



KIET
GROUP OF INSTITUTIONS
Connecting Life with Learning



A Project Synopsis
on
Cybersecurity Awareness & Deepfake Detection Platform
submitted as partial fulfillment for the award of
BACHELOR OF TECHNOLOGY
DEGREE

SESSION 2025-26

in
Computer Science and Engineering

By

Rachit Agarwal(2300290100194)

Raunak Kushwaha(2300290100203)

Pravira Shukla(2300290100183)

Raunak Kumar Bharti(2300290100202)

Under the supervision of

Ms. Deepti Singh

KIET Group of Institutions, Ghaziabad

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow
(Formerly UPTU)

Academic Year 2025-26

Abstract

The rapid growth of web applications has led to an increased attack surface for cyber threats such as SQL Injection (SQLi), Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF). Many small and medium-scale development teams lack accessible, easy-to-use tools to assess the security posture of their web applications during development.

This project proposes an AI-Powered Web Security Analyzer that enables users to submit a website URL or paste/upload a code snippet, and automatically performs basic vulnerability analysis. The system combines rule based static analysis (via regular expressions and pattern matching) with a lightweight machine learning classifier trained on labeled secure and vulnerable code snippets. The analyzer produces a quantified risk score (0-100), categorizes each issue into severity levels (High/Medium/Low), and generates actionable fix recommendations.

The Proof of Concept (PoC), to be developed over 6-8 weeks, will include a React-based frontend dashboard, a Node.js-Express backend with MongoDB for persistence, a Python-based ML model (using scikit-learn) exposed via API, and a PDF report generator. The final system will generate a downloadable report summarizing detected issues, their severity, and recommended mitigation steps. This project demonstrates a practical integration of Cybersecurity and Artificial Intelligence, addressing an important and emerging area of AI-powered cyber defense for web applications.

Introduction

Web applications are at the core of modern digital services, ranging from e-commerce and banking to social platforms and institutional portals. With this increased reliance on web technologies, the number and sophistication of cyber-attacks targeting web applications have also grown significantly. Common vulnerabilities such as SQL Injection, Cross-Site Scripting, and Cross-Site Request Forgery remain prevalent despite the availability of secure coding guidelines and frameworks.

Traditional vulnerability assessment is often performed manually by security experts or through enterprise-grade tools that may be costly, complex, or inaccessible to students, independent developers, and small teams. As a result, many applications are deployed with insufficient security testing, exposing users and organizations to data breaches, unauthorized access, and service disruption.

At the same time, Artificial Intelligence (AI) and Machine Learning (ML) are increasingly being used for pattern recognition, anomaly detection, and automated decision-making in security domains. In the context of software security, ML can assist in learning patterns of insecure code and predicting potential vulnerabilities based on past data. Combining rule-based static analysis with ML-based classification offers a promising hybrid approach, where deterministic checks handle well-known patterns, and ML models generalize to unseen but similar code patterns.

Motivation

The main motivations for this project are:

- To provide an easy-to-use tool for early-stage security assessment of web applications.
- To help students and developers understand and visualize security weaknesses in their own code.
- To explore the integration of ML techniques with classical cybersecurity methods in a practical project.
- To contribute to the growing domain of AI-powered Cyber Defense, which is becoming critical with the rise of AI-driven attacks such as automated exploitation and phishing.

Problem Statement

There is a need for a lightweight, developer-friendly platform that can analyze web application code and endpoints for common vulnerabilities and present the results in a clear, structured, and actionable manner. Such a platform should not only flag potential issues but also provide risk scoring and fix recommendations to guide remediation.

Objectives

The key objectives of the proposed project are:

- To develop a web-based platform where users can submit a URL or code snippet for security analysis.
- To implement basic static vulnerability detection for SQLi, XSS, and CSRF patterns using regex and rule-based checks.

- To design and train a simple ML classifier (e.g., Logistic Regression or Naive Bayes) that predicts whether a code snippet is likely secure or vulnerable based on selected features.
- To compute and present a composite risk score (0–100) and severity levels for detected issues.
- To generate human-readable reports (on dashboard + downloadable PDF) summarizing vulnerabilities, their severity, and recommended fixes.

Scope of Work

The Proof of Concept will focus on:

- Static code and pattern analysis (no full dynamic/runtime analysis in this phase).
- Detection of a subset of vulnerabilities: SQL Injection, XSS, and CSRF indicators.
- A single-user or small multi-user system with basic authentication and scan history.
- Integration of a first-level ML model to complement rule-based detection.

Future extensions, beyond this PoC, can include dynamic scanning, CI/CD integration, more vulnerabilities (OWASP Top 10), and advanced AI-based remediation suggestions.

Literature Survey

Several tools and research works address the problem of web application security assessment and automated vulnerability detection.

Existing Tools

Static Analysis Tools

Tools like SonarQube, CodeQL, and various linters perform static code analysis to detect code smells, potential bugs, and certain security vulnerabilities. They rely primarily on rule sets and pattern matching. While powerful, they often require complex setup, integration into build pipelines, and domain knowledge to interpret findings.

Web Application Security Scanners

Tools such as OWASP ZAP and Burp Suite provide active and passive scanning capabilities against running web applications. They simulate attacks like SQLi and XSS by sending crafted requests. However, these tools can be resource-intensive and may not be ideal for quick, developer-friendly checks at the code level during early development.

Security Plugins and Framework Support

Many web frameworks (e.g., Django, Laravel, Spring) offer built-in defenses and security plugins. While these reduce certain categories of vulnerabilities, misconfiguration and insecure coding practices can still introduce risks, especially when developers bypass or incorrectly use security features.

AI and ML in Vulnerability Detection

Recent research has explored the application of Machine Learning to software security:

- ML classifiers trained on code features (tokens, API calls, control flow patterns) have been used to detect vulnerabilities in source code and binaries.
- Techniques like Naive Bayes, Logistic Regression, Random Forests, and Neural Networks have been applied to classify code snippets as vulnerable or secure.
- Feature engineering often includes the presence of dangerous functions (e.g., eval, exec), use of unvalidated input, string concatenation in queries, and insecure DOM operations such as innerHTML.

These approaches show promise but can be complex to deploy and may require large labeled datasets. For a B.Tech-level project, a lightweight approach using simpler models and carefully selected features is more feasible, especially at PoC stage.

Identified Gaps

From the above survey, the following gaps are identified:

- Many tools are too heavy or complex for small academic or student projects.
- There is a lack of simple, educational tools that clearly bridge the gap between vulnerabilities and concrete code-level fixes.
- Hybrid approaches that combine rule-based detection with a basic ML model are less commonly available in an easy-to-deploy, open, and understandable form.

Proposed Contribution

This project aims to:

- Provide a user-friendly web interface for quick vulnerability checking.
- Combine regex-based static analysis with a small ML classifier, demonstrating a practical hybrid approach.
- Present results using risk scores and severity levels, along with clear recommendations, suitable for students, developers, and academic demonstrations.

Methodology / Planning of Work

The project will be executed in phased steps over 6–8 weeks as follows:

Requirement Analysis and Design

- Finalize list of vulnerabilities (SQLi, XSS, CSRF) and detection rules.
- Design overall system architecture (frontend, backend, ML service, database).
- Define risk scoring formula combining rule-based findings and ML outputs.

Dataset Preparation and Feature Engineering

- Collect or create labeled code snippets categorized as secure and vulnerable.
- Identify and extract features such as presence of eval, exec, innerHTML, direct string concatenation in queries, unescaped input, and missing CSRF tokens.

ML Model Development (Python, scikit-learn)

- Train a baseline classifier (Logistic Regression or Naive Bayes).
- Evaluate using metrics such as accuracy, precision, and recall.
- Expose the trained model using a simple Flask API or a Node–Python bridge.

Backend Development (Node.js + Express)

- Implement /scan endpoint to accept URL or code snippet.
- Integrate regex-based static analysis for SQLi, XSS, and CSRF patterns.
- Call the ML API to obtain vulnerability probability scores.
- Compute overall risk score and classify severity (High/Medium/Low).
- Store scan results in MongoDB.

Frontend Development (React + Tailwind)

- Develop a dashboard with forms to input URL or paste code.
- Display scan status, risk scores, detected issues, and recommendations.
- Provide an option to view previous scans (history).

Report Generation and PDF Export

- Design a structured report template (summary, detailed findings, fixes).
- Implement PDF generation using Puppeteer or pdfkit.

Testing, Validation, and Documentation

- Test the system with multiple sample applications and code snippets.
- Validate correctness of detection logic and ML predictions.
- Prepare user manual, technical documentation, and final presentation.

Facilities Required for Proposed Work

Software Requirements

- Operating System: Windows / Linux
- Frontend: React, Tailwind CSS
- Backend: Node.js, Express.js
- Database: MongoDB (Local or MongoDB Atlas)
- Machine Learning / AI: Python, scikit-learn, Flask
- Other Tools: Puppeteer or pdfkit (PDF generation), Git and GitHub, Visual Studio Code, Postman, Web browser

Hardware Requirements

- Processor: Minimum Intel i5 or equivalent
- RAM: Minimum 8 GB
- Storage: At least 256 GB HDD/SSD
- Network: Internet connection for dependency installation and API testing

References

OWASP Foundation, "OWASP Top 10 – Web Application Security Risks," 2021. Available: <https://owasp.org/Top10/>

F. Massacci, V. Balzarotti, and M. Felderer, "Vulnerability Detection in Web Applications: A Systematic Review," Journal of Systems and Software. available as PDF at: https://www.scirp.org/pdf/JCC_2015091514502611.pdf

F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. Official ACM/portal link: <https://dl.acm.org/doi/10.5555/1953048.2078195>

Node.js Foundation, "Node.js Documentation." Available: <https://nodejs.org/api/index.html>

MongoDB Inc., "MongoDB Manual." Available: <https://www.mongodb.com/docs/>